Java Programming

CPT111 – Lab 4

Teng Ma

Xi'an Jiaotong-Liverpool University
西交利物浦大学

CPT111 Java Programming

Lab 4

For Loop, Arrays, Static Methods 1, and Testing

Testing

- **Test-Driven Development (TDD)** is an iterative process where tests are written ***before*** the actual code is implemented
  - ensure code quality by guiding development through testing
  - leads to simpler, more modular, and better-structured code, since need to think how to break down the problem into testable units

Problem Solving and Testing Example

- Solve this problem and write a function that takes an array and an integer, and returns an integer

QUIZ

Exercise #4.1 Integer Frequency

Quiz | Settings | Questions | Results | Question bank | More

Receive a grade

Opened: Monday, 6 October 2025, 14:00
Closes: Sunday, 19 October 2025, 23:59

Write a Java function/static method `getFrequency(int[] input, int num)` that takes as parameter an array of integers `input` and an integer `num` and returns the *frequency* of appearances of `num` in the array `input`.

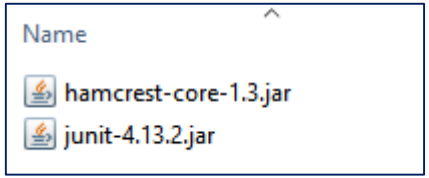
For example, if `input = [1, 2, 1]` and `num = 1`, the function should return 2. And if `num = 2`, the function should return 1.

Testing with JUnit (1)

- **JUnit 4** is one of the most popular frameworks for writing test cases in Java
- Follow the following steps to enable testing with it!

- Download ***two jar files*** from CPT111 course page in Learning Mall

- Save it in a folder where are easy to locate

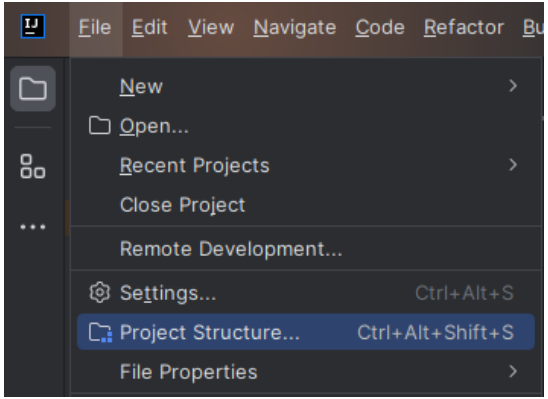


Testing with JUnit (2)

- Create a new project, a class, and the empty static method

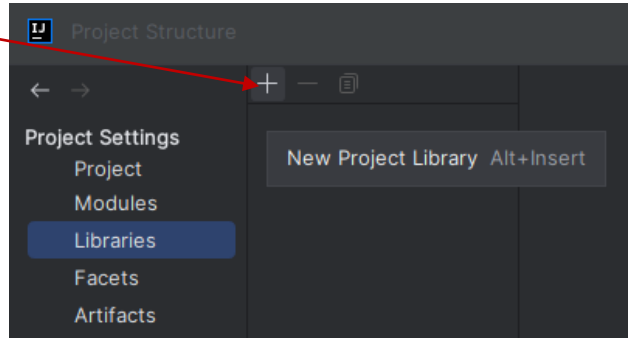
```java
public class Main {
    public static void main(String[] args) { System.out.println("Hello world!"); }

    public static int getFrequency(int[] input, int num) {  no usages

        return 0;
    }
}
```

- Next, we add JUnit 4 library to our project
- Go to File > Project Structure

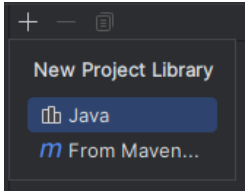


Testing with JUnit (3)

- Go to Libraries > Click Plus Symbol

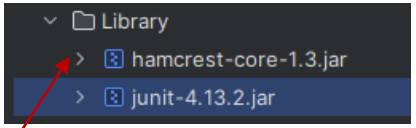


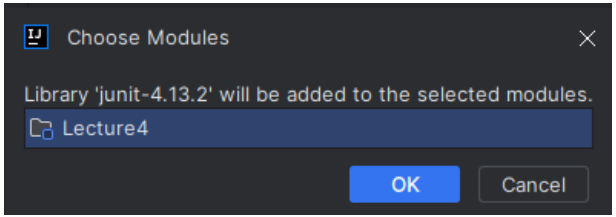
- Select Java



- Select the junit jar file

  and click OK

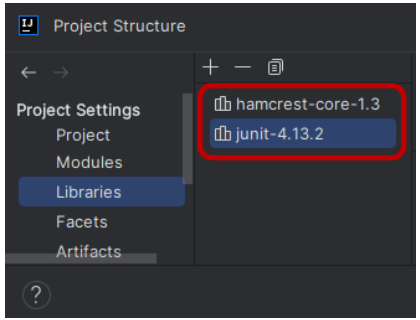




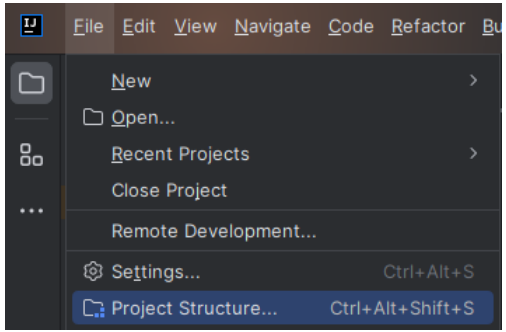
- **<u>Repeat</u>** to add the hamcrest jar file as well!

Testing with JUnit (4)

- JUnit and hamcrest libraries have been added

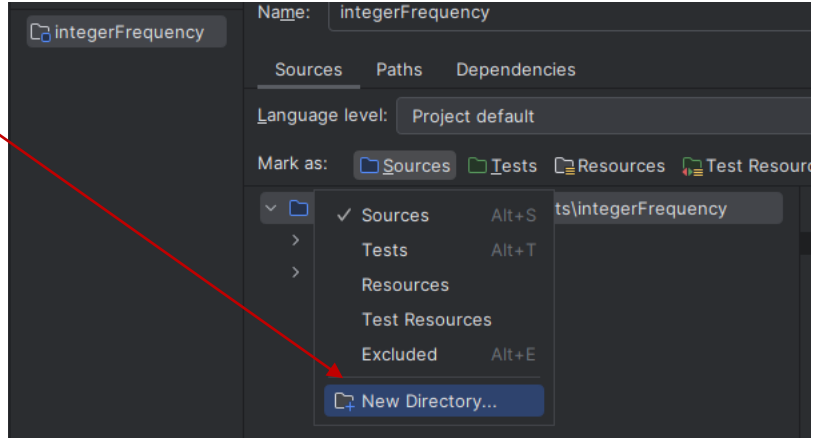


- Next, we are going to add a folder to store our test files
- Go back to File > Project Structures

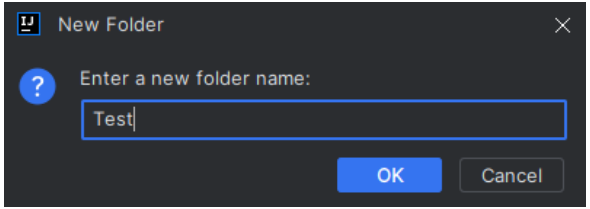


Testing with JUnit (5)

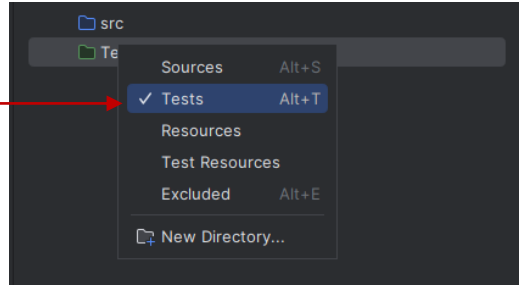
- Select Modules, Right Click Select New Directory...



- Enter a folder name



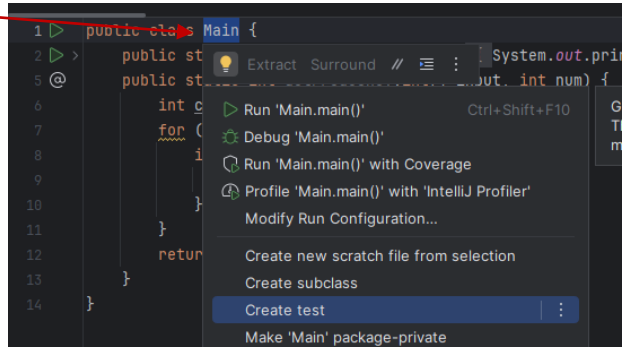
- Right Click and Select Tests

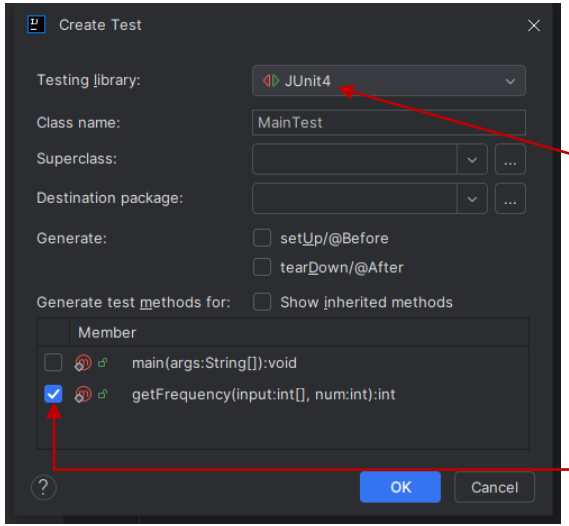


- Click OK

Testing with JUnit (6)

- Now we want to add a test class
- Click the main class name and hit Alt + Enter, select Create Test

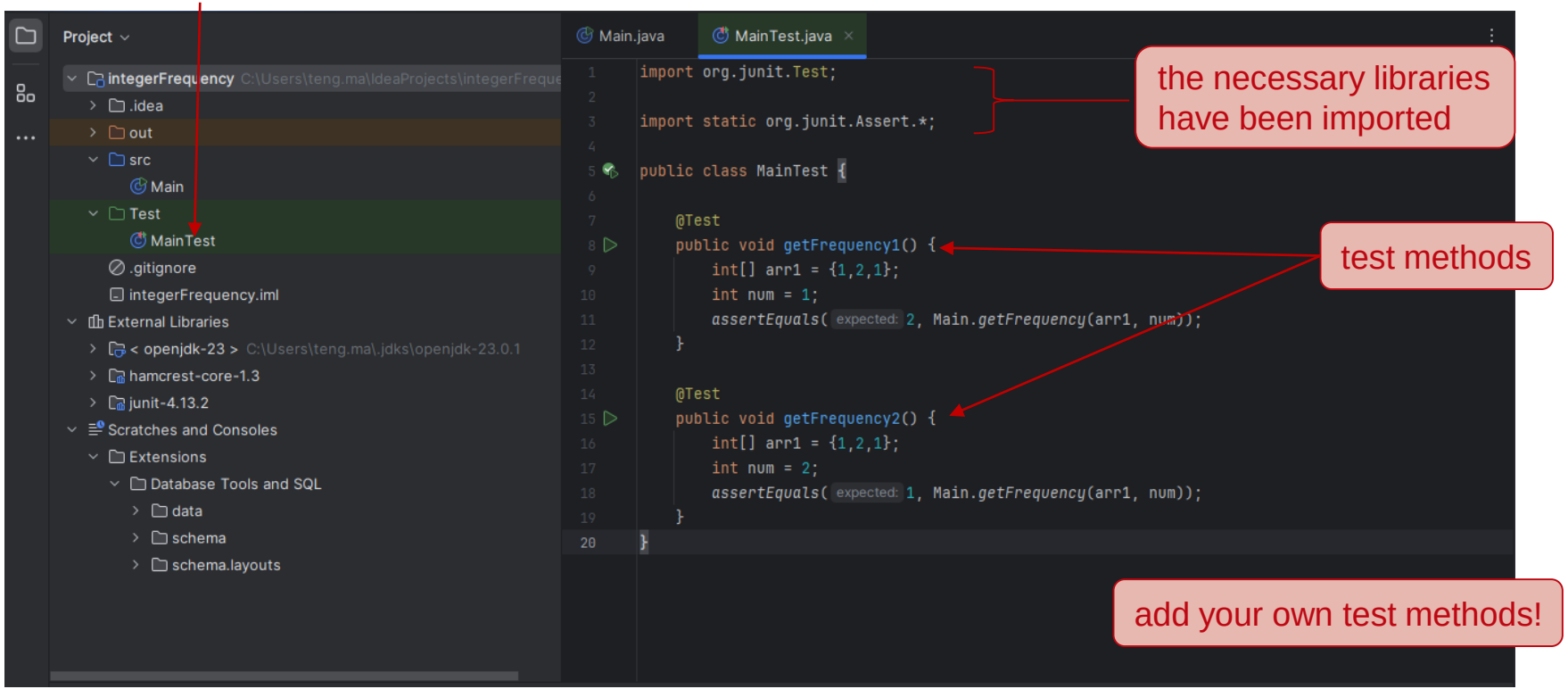




- Choose JUnit4
  and Generate test methods for

Testing with JUnit (7)

- A new test class is added, and finally, we write the **test methods**:



AssestEquals

- **assertEquals** takes two arguments: the **expected** vs. the **actual** values

Main.java | MainTest.java

```java
import org.junit.Test;

import static org.junit.Assert.*;

public class MainTest {

    @Test
    public void getFrequency1() {
        int[] arr1 = {1,2,1};
        int num = 1;
        assertEquals( expected: 2, Main.getFrequency(arr1, num));
    }

    @Test
    public void getFrequency2() {
        int[] arr1 = {1,2,1};
        int num = 2;
        assertEquals( expected: 1, Main.getFrequency(arr1, num));
    }
}
```

click to run all tests

click to run this one test

firstly, write code(s) to produce an actual output

then, compare with the expected correct value

Testing Your Code

- After clicking the green triangle to test your code, you will get

```
MainTest                11 ms
    getFrequency1       10 ms
    getFrequency2        1 ms

C:\Users\teng.ma\.jdks\openjdk-23

java.lang.AssertionError:
Expected :2
Actual   :0
<Click to see difference>
```

since the function only return 0,
it will not match the expected value

- Now complete the code and refactor until you pass all the tests

```
MainTest                4 ms
    getFrequency1       4 ms
    getFrequency2       0 ms

C:\Users\teng.ma\.jdks\openjdk-23.0.1\bin

Process finished with exit code 0
```

Test-driven Development

- Two test cases are given. Suggest more test cases!

Write a Java function/static method `getFrequency(int[] input, int num)` that takes as parameter an array of integers `input` and an integer `num` and returns the *frequency* of appearances of `num` in the array `input`.

For example, if `input = [1, 2, 1]` and `num = 1`, the function should return 2. And if `num = 2`, the function should return 1.

For example:

Test	Result
`int[] input = {1, 2, 1};` `System.out.println(getFrequency(input, 1));`	2
`int[] input = {1, 2, 1};` `System.out.println(getFrequency(input, 2));`	1

Solve the Problem

- After you have passed enough test cases in your own computer, submit to Learning Mall

	Test	Expected	Got	
✔	`int[] input = {1, 2, 1};` `System.out.println(getFrequency(input, 1));`	2	2	✔
✔	`int[] input = {1, 2, 1};` `System.out.println(getFrequency(input, 2));`	1	1	✔

Passed all tests! ✔

Correct

Marks for this submission: 100.00/100.00.

Lab Exercise #4.1

- In this lab, you will learn problem solving using arrays, by creating test cases first

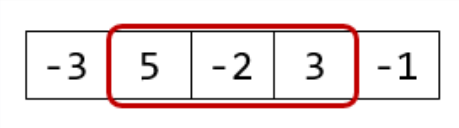
QUIZ

Lab Exercise #4.1 Max Consecutive Sum

- **Complete** the method that takes an integer array and returns an integer:

You are given an array of integers that contains positive, zero, or negative numbers. Your task is to write a static method that finds the **maximum sum** of a sequence of **consecutive elements** in the array and returns this sum.

Write a Java method `int maxConsecutiveSum(int[] arr)` that given an integer array returns the sum of its consecutive elements.



Test cases :

maxConsecutiveSum([-3, 5, -2, 3, -1]) → 6 (the sum of consecutive elements 5 + -2 + 3 = 6 is max)

maxConsecutiveSum([1, -1]) → 1

Lab Exercise #4.1 – Activity 1: Create a Test Class

- Follow the steps we have introduced to **create** a test class
  - by the end, you should have created the following classes:

MaxConsecutiveSum.java × MaxConsecutiveSumTest.java

```java
public class MaxConsecutiveSum {

    public static int maxConsecutiveSum(int[] arr) {


        return 0;
    }

}
```

MaxConsecutiveSum.java MaxConsecutiveSumTest.java ×

```java
import org.junit.Test;
import static org.junit.Assert.*;

public class MaxConsecutiveSumTest {

    @Test
    public void maxConsecutiveSumGivenTestCases() {
        int[] arr1 = {-3, 5, -2, 3, -1};
        int output1 = MaxConsecutiveSum.maxConsecutiveSum(arr1);
        assertEquals( expected: 6, output1);

        int[] arr2 = {1, -1};
        int output2 = MaxConsecutiveSum.maxConsecutiveSum(arr2);
        assertEquals( expected: 1, output2);
    }

}
```

Lab Exercise #4.1 – Activity 2: Create a good set of test cases

- **Add** more test cases to the test class
  - use meaningful test method names

```java
public class MaxConsecutiveSumTest {

    @Test
    public void maxConsecutiveSumGivenTestCases() {
        int[] arr1 = {-3, 5, -2, 3, -1};
        int output1 = MaxConsecutiveSum.maxConsecutiveSum(arr1);
        assertEquals( expected: 6, output1);

        int[] arr2 = {1, -1};
        int output2 = MaxConsecutiveSum.maxConsecutiveSum(arr2);
        assertEquals( expected: 1, output2);
    }

}
```

- Here are some ideas:
  - all positives
  - all negatives
  - all zeros
  - empty array
  - array with one element or two elements
  - array with mix values of large positive and negative numbers
  - array with a single element sum located at the beginning/middle/end
  - ...

Lab Exercise #4.1 – Activity 3: Solve the Problem

- Write your code and continuously refactor it to pass more test cases
  - a simple solution using two nested for-loops is possible
  - *extra challenge:* can you solve the problem with *just* one loop?

- Once you have passed all of your test cases, submit to Learning Mall
  - if your code fails, add more test cases and debug it!

	Test	Expected	Got	
✔	`int[] arr = {-3, 5, -2, 3, -1};` `System.out.println(maxConsecutiveSum(arr));`	6	6	✔
✔	`int[] arr = {1, -1};` `System.out.println(maxConsecutiveSum(arr));`	1	1	✔

Passed all tests! ✔

Correct

Marks for this submission: 100.00/100.00.

Lab 4 Challenge – Sum of Proper Elements

- In the lab challenge, you will repeat the activities in the lab exercise with a similar problem using an array, method, and testing strategies:

QUIZ

Lab 4 Challenge - Sum of Proper Elements

- **Complete** the method that takes an integer array and returns an integer:

We say that an element of an integer array is **proper** if and only if it is some multiple of (its index + 1).

Write a Java method `int sumOfProper(int[] nums)` that given an integer array returns the sum of all its proper elements.

Test cases :

sumOfProper([1, 2, 6]) → 9 (from 1 + 2 + 6, since all elements are proper)

sumOfProper([10, 25]) → 10 (since only 10 is a proper element)

Lab 4 Challenge – Task 1: Create a Test Class (20 marks)

- Follow the steps in Lecture 4, slides 20 – 26 to **create** a test class
  - by the end, you should have created the following classes:

SumOfProperElements.java × SumOfProperElementsTest.java

```java
public class SumOfProperElements {

    public static int sumOfProperElements(int[] nums) {


        return 0;
    }

}
```

SumOfProperElements.java SumOfProperElementsTest.java ×

```java
import org.junit.Test;
import static org.junit.Assert.*;

public class SumOfProperElementsTest {

    @Test
    public void sumOfProperElements() {



    }
}
```

Lab 4 Challenge – Task 2: Create a Set of Test Cases (40 marks)

- **Add** more test cases to the test class
  - use meaningful test method names
  - try to include cases that cover all possible output

```java
@Test
public void sumOfProperElementsGivenTestCases() {
    int[] arr1 = {1, 2, 6};
    int output1 = MaxConsecutiveSum.maxConsecutiveSum(arr1);
    assertEquals( expected: 9, output1);

    int[] arr2 = {10, 25};
    int output2 = MaxConsecutiveSum.maxConsecutiveSum(arr2);
    assertEquals( expected: 10, output2);
}
```

add your own test cases in new methods below!

Lab 4 Challenge – Task 3: Solve the Problem (40 marks)

- Write your code and continuously refactor it to pass more test cases
- Once you have passed all of your test cases, submit to Learning Mall
  - if your code fails, add more test cases and debug it!
- **Show** to your Lab TA that you have passed the Learning Mall Quiz:

	Test	Expected	Got	
✔	`int[] nums = {1, 2, 6};` `System.out.println(sumOfProper(nums));`	9	9	✔
✔	`int[] nums = {10, 25};` `System.out.println(sumOfProper(nums));`	10	10	✔

Passed all tests! ✔

Correct

Marks for this submission: 100.00/100.00.

- **Briefly explain** to the TA you solution!

Thank you for your effort!

- If you finish early, feel free to use the remaining time to work on other online exercises
  - use this opportunity to practice your new skills in testing your code!

- You may leave the lab once you have complete the Lab 4 Challenge and it has been ASSESSED by a TA!
  - Please note that the **Lab 4 Challenge** is **graded** and contributes to your CW2 marks